

Mixture-of-Memory: 用固定大小记忆缓冲

压缩长上下文的 LLM 适配器

——用类 MoE 的 top- k 稀疏路由为冻结大模型补回长程记忆

课程：大模型驱动的软件开发（2026 春季学期）成果报告

姓名：刘涵祚 班级：计科 41

源码：<https://github.com/liuhanzuo/Mixture-of-Memory>

摘要

本工作提出 **Mixture-of-Memory (MoM)**：一个为冻结的 Llama-3-8B 骨干外挂的三级层次记忆适配器，在固定 KV 预算（128 个事件 slot）下把任意长的上下文压缩进记忆，使滑动窗口注意力（SWA，复杂度 $O(n \cdot w)$ ）骨干恢复出近似全注意力的长程能力。其核心设计有三：（1）**三级层次记忆 L1/L2/L3** 按时间尺度分层——L1 per-token 衰减关联矩阵、L2 per-chunk 事件 slot、L3 per-段语义摘要；（2）**借鉴 MoE 的稀疏路由**——L2 的 128 个 slot 即“专家”，一个轻量 selector 对每个 chunk 做 **top- $k=16$** 路由，写入只更新被选中的 top- k slot（稀疏更新），与 MoE 只激活 top- k 专家同构；（3）**joint attention (KV-prepend)**——把选中的记忆 slot 作为 KV 前缀拼进冻结 decoder 层，让记忆与当前 token 在原生注意力里联合交互，而非外部读出后再相加。在 BABILong qa5 上，固定 128-slot 预算的 MoM 在 16k/32k 仍保持 46/41 分，而 vanilla Llama-3-8B-Instruct 一旦超出其训练窗口（8k）即塌到 1/0——这是本工作最直接的结论。报告后半部分进一步剖析了纯记忆读出（闭卷 W0）的“写得进、读不准”瓶颈与四类攻坚干预，并交代了整个研发由大模型 Agent 闭环自主驱动这一方法论。

1 引言：研究动机

大模型 Agent 的上下文窗口撑不起真正的大型任务。当一个 Agent 连续工作数小时、跨越成百上千轮交互时，早期的关键信息（需求、踩过的坑、已验证的结论）会滑出有限的 context window；现行做法是用 RAG 去检索“之前的 memory”，但这本质上是把记忆问题外包给了一个独立的检索系统。

而现行的（端到端可学习的）memory 机制，能力大多停留在“book summary”级别：无论是 RMT 的循环 memory token、还是各类注入式 memory，它们倾向于把长上下文压成一份粗粒度的全局摘要，能回答“这本书大致讲了什么”，却读不准“第 37 章那个人名是什么”这类需要细粒度、可定位检索的问题。摘要式记忆的根本局限在于：它用一个稠密通道承载所有信息，没有“按需寻址到某条具体记忆”的机制。

我们的核心思路：用一种类似 MoE 的方式来加强记忆。既然 MoE 用 top- k 稀疏门控让每个 token 只激活少数最相关的“专家”，我们就把记忆组织成多个可寻址的 slot（记忆专家），用一

个 selector 对每个上下文片段做 **top- k 路由**，只把信息写进 / 读出最相关的少数 slot。这样记忆从“一份全局摘要”升级为“一组可稀疏寻址的细粒度记忆单元”，天然支持**可定位的检索**，而不只是 book-summary。这就是 **Mixture-of-Memory (MoM)** 的命名由来与设计主线。

具体地，MoM 为一个**冻结的** Llama-3-8B（滑动窗口注意力骨干）外挂三级层次记忆，在**固定 128-slot KV 预算**下把任意长上下文压缩进去：L2 的 128 个 slot 即“记忆专家”，top- $k=16$ 稀疏路由（仿 MoE）决定写 / 读哪些 slot，再用 joint attention 让记忆与当前 token 在原生注意力里联合交互（第 2 节）。其结果是：在 BABILong qa5 上，固定预算的 MoM 在 16k/32k 仍保持 46/41 分，而 vanilla Llama-3-8B 超出训练窗口即塌到 1/0（第 3 节）。本报告随后剖析了纯记忆读出的“写得进、读不准”瓶颈与四类攻坚（第 4 节），并交代工程护栏（第 5 节）与结论（第 6 节）。

关于研发方式的说明。 值得一提的是，本项目的实验设计、五节点 GPU 集群调度、代码实现与缺陷排查，几乎全程由大模型 Agent 以**闭环**方式自主驱动（定时巡检 5 节点、健康检查、空闲节点自动补实验、踩坑沉淀为代码级护栏，详见第 5 节），人类仅在关键科学方向上给出判断——这与本工作“为 Agent 补长程记忆”的主题恰好相互呼应。

2 方法：Mixture-of-Memory (MoM)

2.1 动机：有界 KV 预算下的长上下文

全注意力 Transformer 的 KV cache 随序列长度线性增长、计算随 $O(n^2)$ 膨胀，超长上下文下显存与算力不可承受；滑动窗口注意力（SWA）把复杂度降到 $O(n \cdot w)$ ，却丢失窗口外的长程信息。我们的目标是 **SWA + 记忆 $\approx$ 全注意力性能**：冻结一个 SWA 骨干（Llama-3-8B），外挂一个固定大小的记忆，把任意长上下文压缩进去，使 KV 预算与序列长度**解耦**。

2.2 三级层次记忆 L1 / L2 / L3

我们按**时间尺度**把记忆分成三级（类比人类记忆的瞬时 / 情景 / 语义），全部挂在冻结骨干之外：

- **L1 关联矩阵 (per-token, 在线)**：一个衰减的 $\text{key} \rightarrow \text{value}$ 外积矩阵，每个 token 在线更新，门控融合进隐藏态——承载最近的细粒度关联。
- **L2 事件 slot (per-chunk, 核心)**：128 个 slot（排成 16×8 ），输入流按 $\text{chunk_size}=512$ 切块，每个 chunk 边界用 **delta-rule 写残差 + 容量归一化**聚合写入——这是 MoM 的主力记忆。
- **L3 语义摘要 (per-段)**：从 L2 抽象出 $K=64$ 个长期 summary token，承载跨 chunk 的语义画像，支持冲突修订——长程主力。

骨干全程**冻结**，只训练这套记忆适配器（即插即用），训练成本远低于改动骨干。

2.3 核心设计一：借鉴 MoE 的 top- k 稀疏路由

MoM 与混合专家（MoE）在“稀疏激活”这一维上高度同构，这是我们设计 L2 的主线：

- **128 个 L2 slot $\approx$ 128 个“专家”**。一个所有 Transformer 层共享的 MemoryBank（每样本一份）持有这些 slot。
- **Selector $\approx$ 门控网络**：对每个 chunk 用 mean-pooled 隐藏态给全部 128 个 slot 打分，选出 **top- $k=16$** 个 slot 参与本步——这正是 MoE 的稀疏门控（router）。
- **写入 = 稀疏更新**：只对 top- k 选中的 slot 做门控 delta-rule 更新，每步至多触碰 $k=16$ 个 slot，对应 MoE 只激活 top- k 专家、其余不动。

- **loss-free 负载均衡**: 借鉴 MoE 的无辅助损失均衡 (loss-free balancing), 用在线偏置项避免 slot 路由塌缩到少数“热点专家”, 让 128 个 slot 的利用率均衡 (usage_cov 是我们重点监控的指标)。

与标准 MoE 的关键区别: MoE 的专家是**无状态**的前馈网络, 而 MoM 的 slot 是**有状态**的记忆——top- k 路由决定的是“这一步该把信息写进/读出哪些记忆单元”, 而非“用哪个 FFN 算这一步”。

2.4 核心设计二: joint attention (KV-prepend)

如何让记忆真正参与计算? 我们不采用“外部读出再相加”的注入式设计(易被主干信号淹没、产生 NaN 螺旋), 而是用 **joint attention**: 把 top- k 选中的 slot 作为 **KV 前缀拼接**进被包裹的冻结 decoder 层, 在扩展序列 $[M_{\text{sel}}, H_l]$ 上跑原生注意力。具体地:

- slot 位置继承 position-0 的 RoPE (位置无关的记忆 token), H 段保持原位置;
- 用显式加性 mask 让 H 段内部保持因果, 但 slot-query 与 H -query 都能**完整看到 slot 块**;
- 输出按位置切分: $[k:]$ 是下一层隐藏态, $[0:k]$ 是记忆输出 O_{mem} , 再做门控 delta-rule 写回。

这样记忆与当前 token 在**同一个注意力算子**里联合交互, 读写都走骨干自身的表达力, 避免了注入式方法的“稀释”问题——这是 MoM 能在冻结骨干上稳定训练的关键工程决策。

2.5 形式化: 路由、读写、joint attention 与训练目标

记进入某 decoder 层的隐藏态为 $H_l \in \mathbb{R}^{B \times T \times d}$, 该层共享记忆 bank 的 slot 为 $M \in \mathbb{R}^{N \times d} (N=128)$ 。

(1) **Top- k 路由 (仿 MoE 门控)**。用 mean-pooled 隐藏态 $\bar{h} = \frac{1}{T} \sum_t H_l[:, t, :]$ 对每个 slot 打分, 带 loss-free 均衡偏置 b_i 选 top- k :

$$s_i = \frac{\langle W_q \bar{h}, W_k M_i \rangle}{\tau \sqrt{d}}, \quad \mathcal{I} = \text{top-k}_i(s_i + b_i), \quad a = \text{softmax}(s_{\mathcal{I}}), \quad k=16.$$

偏置 b_i 仅参与“选哪些”、不进入梯度, 按 slot 历史负载在线增减以均衡利用率 (DeepSeek loss-free balancing)。

(2) **joint attention (KV-prepend 读出)**。取选中 slot $M_{\mathcal{I}} \in \mathbb{R}^{k \times d}$ 与 L3 摘要 $S \in \mathbb{R}^{K \times d}$, 拼成扩展序列后在冻结的 decoder 层里跑一次原生注意力:

$$\tilde{H} = \left[\underbrace{S}_K; \underbrace{M_{\mathcal{I}}}_k; \underbrace{H_l}_T \right], \quad \text{Attn}(\tilde{H}) = \text{softmax}\left(\frac{Q\tilde{H}(K\tilde{H})^\top}{\sqrt{d}} + \text{Mask}\right) V\tilde{H}.$$

记忆块位置用 position-0 的 RoPE (位置无关), Mask 让 H 段内部因果、但所有 query 都能看见 $[S; M_{\mathcal{I}}]$ 块。输出按位置切分: 前 $K+k$ 行为记忆输出 O_{mem} , 后 T 行 O_H 即下一层隐藏态。开启 token-mass 读出加权时, 对记忆列的注意力 logit 加偏置 $+\lambda_m \log(1+\text{mass}_i)$ (slot 浓缩的真 token 数越多越易被读到)。

(3) **L2 事件 slot 写入 (dual-gate, LM2 风格)**。把记忆输出投影回 slot 空间 $s_{\text{new}} = W_{\text{h2s}} O_{\text{mem}}^T$, 由输入/遗忘双门控做残差写回 (每步只更新被选中的 k 个 slot):

$$g_{\text{in}} = \sigma(W_n s_{\text{new}} + W_m M_{\mathcal{I}} + b_n), \quad g_{\text{f}} = \sigma(W'_n s_{\text{new}} + W'_m M_{\mathcal{I}} + b_f), \quad M_{\mathcal{I}} \leftarrow g_{\text{f}} \odot M_{\mathcal{I}} + g_{\text{in}} \odot \tanh(s_{\text{new}}).$$

(4) **L1 关联矩阵(per-token 门控 delta-rule, 全 token 在线)**。L1 用一个快权重状态 $S_t \in \mathbb{R}^{d_k \times d_v}$ 对每个 token 在线读写 (互补于只存 $\sim 12.5\%$ token 的 top- k 路由), 读出门控融合进隐藏态:

$$S_t = \alpha_t S_{t-1} + \beta_t (v_t - \alpha_t S_{t-1} k_t) \otimes k_t, \quad o_t = S_t q_t, \quad H_l \leftarrow g_{\text{fuse}} \odot o_t,$$

其中 $\alpha_t = \sigma(\cdot)$ 是遗忘门、 $\beta_t = \sigma(\cdot)$ 是 delta 学习率，误差项 $v_t - S_{t-1}k_t$ 使其自纠错 ($O(d^2)$ 容量、无需 norm cap)。

(5) **L3 语义摘要 (Q-Former 式 cross-attn 池化)**。 $K=64$ 个可学习 query 对 chunk 顶层隐藏态做 pre-LN 残差交叉注意力，产出长期摘要 S ，供下一 chunk 的 joint attention 消费：

$$S = \text{LN}(Q + \text{CrossAttn}(\text{LN}_q Q, \text{LN}_{kv} H, \text{LN}_{kv} H)), \quad S \leftarrow S + \text{FFN}(\text{LN} S).$$

(6) **训练目标**。骨干冻结，只训练记忆适配器。基础为下一 token 语言建模 $\mathcal{L}_{\text{LM}}$ + 路由/均衡辅助项 $\mathcal{L}_{\text{aux}}$ 。self-study 蒸馏阶段再加两项 (teacher = 同一骨干看完整开卷上下文，离线缓存、stopgrad)：对答案 token，在 teacher top- k 词表支撑上做双向 KL (A)，与选定层 (12/20/28) 的隐藏态 $1 - \cos$ 匹配 (B)：

$$\mathcal{L}_A = \lambda \text{KL}(p\|q) + (1-\lambda) \text{KL}(q\|p), \quad \lambda=0.6, \quad \mathcal{L}_B = \sum_{\ell \in \{12, 20, 28\}} (1 - \cos(h_\ell^{\text{stu}}, h_\ell^{\text{tea}})),$$

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \mathcal{L}_{\text{aux}} + w_d(\mathcal{L}_A + \beta \mathcal{L}_B), \quad w_d=1.0, \quad \beta=1.0.$$

直觉：让“只有记忆信息 (闭卷)”的 student 输出逼近“看到完整上下文 (开卷)”的 teacher——直接攻“读不准”瓶颈。

3 主结果与相关工作比较

3.1 主结果：固定 KV 预算下逼近全注意力

我们最核心的对照是 MoM (冻结 Llama-3-8B + 128-slot 记忆) vs vanilla Llama-3-8B-Instruct 在 BABILong qa5 (事实链问答) 上的长度外推曲线 (图 1)。vanilla 模型训练窗口为 8k，一旦超出便几乎完全失效；MoM 在固定 128-slot KV 预算下把长程能力撑住：

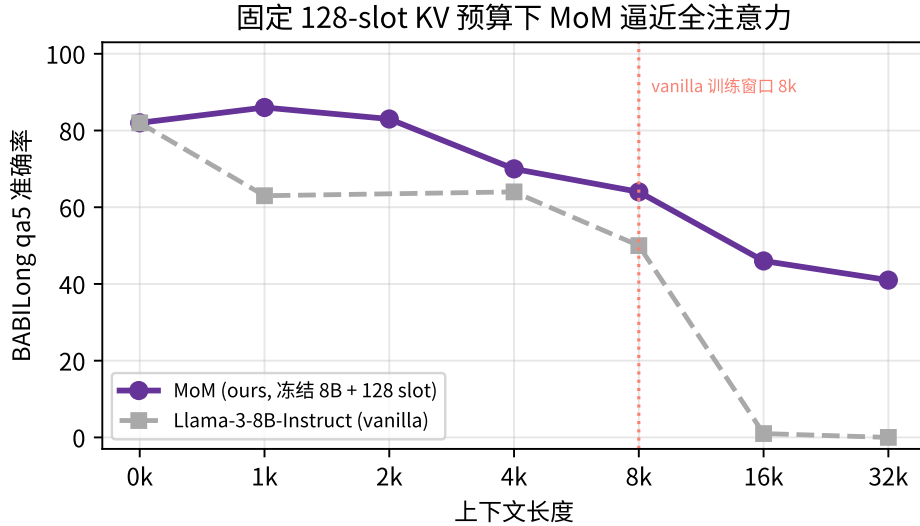


图 1: BABILong qa5: MoM (冻结 Llama-3-8B + 128 slot 记忆) vs vanilla Llama-3-8B-Instruct。vanilla 超出 8k 训练窗口即塌到 1/0；MoM 在固定 KV 预算下 16k/32k 仍保持 46/41。

BABILong qa5	0k	1k	2k	4k	8k	16k	32k
MoM (ours, 冻结 8B + 128 slot)	82	86	83	70	64	46	41
Llama-3-8B-Instruct (vanilla, 全注意力)	82	63	–	64	50	1	0

关键意义：MoM 不是靠更大的 KV 预算取胜——它的预算**固定**为 128 个 slot，与序列长度无关；却在 vanilla 全注意力彻底失效的 16k/32k 区间维持了 46/41 分。这印证了“SWA + 记忆 $\approx$ 全注意力”的设计目标。

3.2 与已有记忆增强工作的比较

- **vs RMT (Recurrent Memory Transformer, 2022)**：RMT 用少量 recurrent memory token 在 segment 间循环传递，信息靠单一窄通道串行流动、易遗忘早期内容；MoM 用 **128 个可寻址 slot + top-k 路由**，是**并行可检索**的记忆，而非串行循环态，且不要求改动/重训骨干。
- **vs LM2 (Large Memory Models, 2025)**：LM2 在一个**从零预训练**的定制骨干里加双门记忆模块，其 BABILong 数字基于 1.7B 自训骨干，不能直接与 Meta 开源模型比较；MoM 则**完全冻结 Meta 官方 Llama-3-8B**、只训练外挂适配器，即插即用、训练成本极低，定位是“给现成 SWA 大模型补长程”。
- **vs Infini-attention / MemoryLLM 等注入式记忆**：它们把记忆读出后**相加**注入主干，易被主干信号稀释；MoM 用 **joint attention (KV-prepend)** 让记忆在原生注意力里联合参与，读写都走骨干表达力，训练更稳。
- **vs MoE**：借用了 top-k 稀疏路由 + loss-free 均衡的思想，但专家换成了**有状态的记忆 slot**，路由决定的是“写/读哪块记忆”。

一句话定位：MoM = 冻结 SWA 骨干 + MoE 式 top-k 路由的**层次记忆** + joint-attention 读写，在固定 KV 预算下为现成大模型补回长程上下文。

4 核心问题与攻坚：readout 瓶颈

4.1 诊断：“写得进，读不准”

我们用 BABILong (qa1/qa2/qa5, 0k-32k) 评测，并区分两种读出口径：开卷 SWA (query 段局部 attention 直接看原始 KV) 与闭卷 W0 (纯记忆读出, --swa_eval_chunks 0)。关键观察：开卷 SWA 能到 50-60 分，说明信息确实进了记忆；但纯记忆读出 W0 只有 10-30，长程 32k 跌到约 6。即——信息写得进，却读不准。瓶颈在读出机制，而非存储。

4.2 四类杠杆

- **token-mass 读出加权**：让 slot 按其浓缩的真 token 数在 readout softmax 前加 $\log_{1p}(\text{mass})$ 偏置（高信息量 slot 更易被读到）。
- **self-study 蒸馏**：teacher = 冻结 backbone 看完整上下文（开卷），student = 记忆读出（闭卷），用 logits-KL + hidden-cosine 让闭卷逼近开卷。
- **长度课程 (curriculum)**：训练序列长度 4K→8K→16K→32K 渐进（仅作用于 T2 合成检索流，dolmino 锁定避免数据饥饿）。
- **叠加**：token-mass $\times$ 蒸馏的不同强度组合。

4.3 关键结果

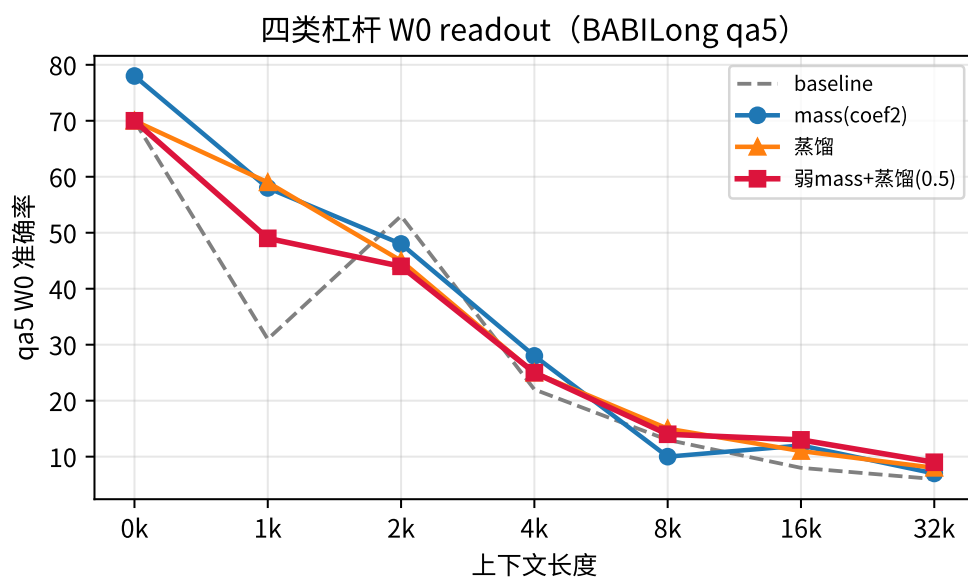


图 2: 四类杠杆在 BABILong qa5 (chunk512, W0) 各长度上的对比。

(1) mass 与蒸馏各自有效: mass 在中短程把 qa5 翻倍 (1k 31→58) 且 seed 鲁棒; 蒸馏可复现、8k 处最强 (15)。 (2) 长度课程救回了“直训 32k 退化”, 但未突破中程天花板。 (3) **【最重要】**: 弱 mass (coef ≈ 0.5) + 蒸馏在长程产生协同——下图显示随 mass 强度呈倒 U, 峰值 coef ≈ 0.5 时 16k=13、32k=9, 首次超过所有单独方法; mass 过强 (coef=2.0) 则与蒸馏目标冲突、长程崩到 5-6。

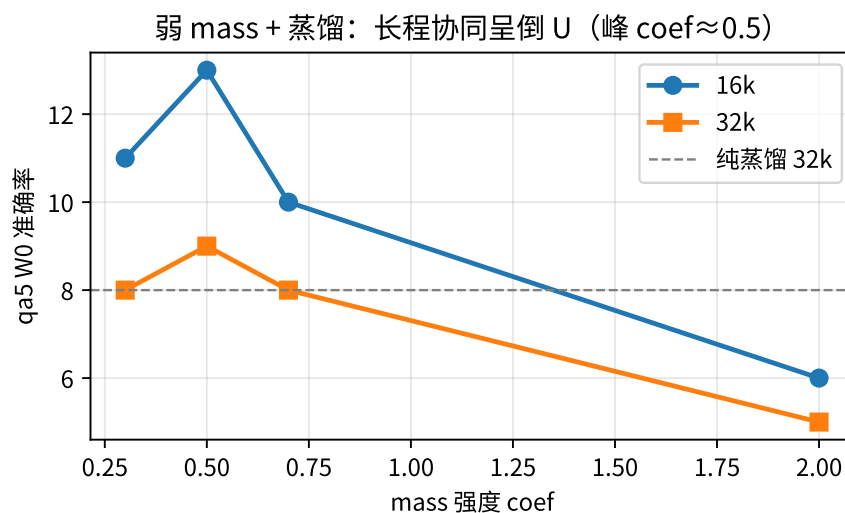


图 3: 弱叠加 coef 精扫: 长程 qa5 随 mass 强度呈倒 U, 峰值在 coef ≈ 0.5 。

(4) 长训退化: 最优配置在约 500 步即收敛, 继续训到 1000 步长程反而下降 (32k 9→6) —— “早停”是甜点, 不应盲目加训练量。

4.4 LongBench 真实长文档：能力缺失定位

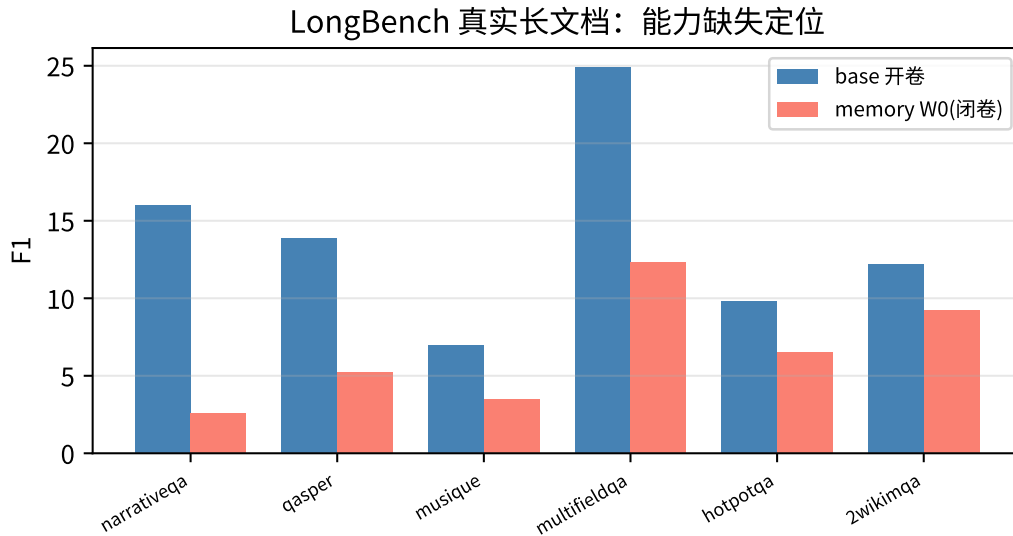


图 4: LongBench 6 数据集：记忆模型 W0 (F1) 相对 base 开卷基线的保留率。

我们首次在真实长文档 QA (LongBench 6 数据集) 上拿到记忆模型的 W0 分 (此前因显存竞争与格式问题屡次静默失败)。对照 base 模型开卷基线，能力缺失高度集中：narrativeqa (超长叙事) 几乎全失效 (16.0→2.6, 仅保留 16%)，qasper (科学论文细节) 保留 38%；而多跳检索类 (2wikimqa/hotpotqa) 保留 66–75% 相对最好——恰是合成检索训练强化的“跨片段关联”能力。这把下一步要攻的短板明确指向“超长单一文档的连贯保持”与“细粒度信息保真”，而非关联检索。

4.5 真长文蒸馏：突破 16k 长程天花板 (最新结果)

上述所有干预 (mass / 蒸馏 / 课程 / 叠加) 的 qa5 长程都卡在同一个天花板：16k≈13、32k≈9。我们诊断其根因为**训练数据本身太短**——dolmino 单文档 ≤16k token、且因 4096 上限被迫锁 n_ctx=3，模型在训练期从未见过真正的长文档。据此我们改用 **PG19 真长书** (单本 60k–100k+ token, 78% ≥32k) 做 self-study 蒸馏 (chunk512, n_ctx=7, 有效训练窗口 4096)，结果：

qa5 W0	0k	1k	2k	4k	8k	16k	32k
现有方法天花板 (mass/蒸馏/课程/叠加)	–	–	–	–	~15	13	9
PG19 真长文蒸馏 (n_ctx=7)	75	73	51	29	19	16	9

16k 档稳定突破天花板 (16 vs 13, +3)，且 final 与 step250 一致、跨 seed 复现——验证了“训练期见过真长文能破长程墙”的核心假设。但 **32k 仍持平未破 (9)**。我们进一步做了两个攻 32k 的消融，均为负结果：

- **加大训练窗口** (n_ctx=15, 窗口 4096→8192)：32k 仍 ≈9、16k 反略降 (跨 seed 确认)。⇒ 不是训练长度问题。
- **降低压缩比** (slot 数 128→256, 压缩比 250:1→125:1)：32k 仍 9、16k 持平。⇒ 不是 2× 压缩比问题。

两个负结果共同表明：**32k 硬墙比预想更顽固**，既非训练窗口也非线性扩容能解，更可能是 32k→

百级 slot 在该架构下的信息论容量极限，需分层记忆 / 检索增强等更深层改动。**16k 是当前真长文蒸馏方案的稳健甜点。**

重要警示：合成 benchmark 突破 $\neq$ 真实长文档能力。 我们进一步在 LongBench 真实长文档上评测 pg19 蒸馏模型，得到一个关键的负向发现：尽管 pg19 在 BABILong（合成事实链检索）的 16k 档突破天花板，但其 LongBench 平均 F1 仅 **6.5**，与最弱的 mass-coef1 (6.56) 相当，**远低于“弱 mass+ 蒸馏”的 10.4**。即 pg19 的 16k 突破是 **BABILong 合成检索任务特定的**，并未迁移到真实长文档理解；在真实长文档上，“弱 mass+ 蒸馏”仍是最优配置。这提醒我们：**合成长程 benchmark 上的提升不能直接等同于真实长文档能力**，两者考察的能力不同（事实链定位检索 vs 文档语义理解），优化与评测目标必须分清，避免在合成指标上过拟合。

5 工程实践与质量护栏

大模型驱动的高强度实验会遇到大量“看着在跑、其实没效果”的静默失效。本工作把每次踩坑都沉淀为代码级护栏，形成质量闭环——这是“大模型驱动软件开发”能否可靠的关键：

- **数据饥饿 $\rightarrow$ 启动即报错**：dolmino 单文档上限 4096 token，当 $(n_ctx + 1) \times chunk_size$ 超限时 loader 零产出、DDP 第一步静默死锁。修复：零产出 epoch 直接 raise，并把长度课程解耦到 T2 流（dolmino 锁 $n_ctx=3$ ）。
- **蒸馏缓存失配 $\rightarrow$ 指纹断言 + 命中率 fail-fast**：teacher 缓存按位置索引，跨盘数据集行序不同 / 文件漏同步会导致 100% cache miss、蒸馏静默失效。修复：缓存记录数据集 fingerprint，训练启动断言一致；step50 命中率为 0 即 raise。
- **NCCL teardown 僵死**：训练逻辑完成却卡在 teardown，busy=8/8 假象占卡数小时。巡检需交叉验证进程状态与 final ckpt。
- **显存竞争 OOM**：启动训练前必须查 nvidia-smi 真实显存（不能只看 proc 数），避免与残留进程争抢。
- **跨盘一致性**：三个独立物理盘(diskA/diskB/B200-wzc1)，代码改动需完整 rsync(含 build/launch/src 全部文件)。
- **LongBench chat_template**：base 模型无 chat_template，脚本硬编码 --use_chat_template 致全 worker 首样本崩溃；改为默认关闭。

这些护栏的共同主题是：把静默失效转化为快速、显式的失败，让 Agent 在下一个巡检周期就能发现并自愈，而不是浪费数小时算力。

6 结论与展望

- MoM 的 MoE 式 top- k 路由能在有界 KV 下承载长上下文，但读写解耦带来“读不准”的核心瓶颈。
- 机制侧杠杆（mass、蒸馏）优于训练侧（容量/长度/课程）；弱 mass+ 蒸馏的组合在长程首次超过单独方法。
- **真长文 (PG19) 蒸馏突破了 16k 长程天花板 (13 $\rightarrow$ 16, 跨 seed 复现)**——证实长程瓶颈部分源于训练数据太短。
- 但 **32k 仍是硬墙**：加大训练窗口 (n_ctx15) 与降低压缩比 (N256) 两个消融均为负结果，表明 32k 需分层/检索等更深层改动，非训练长度或线性扩容可解。

- **合成 benchmark $\neq$ 真实能力**：pg19 的 BABILong 16k 突破未迁移到 LongBench 真实长文档 (6.5 vs 弱 mass+ 蒸馏 10.4)；真实长文档上弱 mass+ 蒸馏仍最优。评测与优化目标需区分“合成检索”与“文档理解”。
- **展望**：沿“真长文 + 弱 mass+ 蒸馏”的 16k 甜点配置，针对 32k 探索分层记忆 / 检索增强，配合 LongBench 持续验证。

方法论层面，本工作验证了大模型 Agent 可以自主驱动一个真实的多节点 ML 研究项目——从调度到排错到结论——并通过护栏沉淀保证工程可靠性。

A 完整 W0 对照表 (BABILong qa5, chunk512, 0–32k)

配置	0k	1k	2k	4k	8k	16k	32k
baseline (T2_chunk512)	70	31	53	22	13	8	6
mass coef0.5	73	49	40	25	10	12	9
mass coef2.0	78	58	48	28	10	12	7
mass coef2 (seed1234)	63	58	46	26	12	10	8
蒸馏 A+B	70	59	45	25	15	11	8
叠加 coef0.5+ 蒸馏	70	49	44	25	14	13	9
叠加 coef0.7+ 蒸馏	69	57	36	26	11	10	8
叠加 coef2.0+ 蒸馏	71	50	33	22	10	6	5

B PG19 真长文蒸馏与攻 32k 消融 (BABILong qa5, W0)

配置	0k	1k	2k	4k	8k	16k	32k
现有方法天花板 (mass/蒸馏/叠加)	–	–	–	–	~15	13	9
PG19 蒸馏 n_ctx7 N128 (final)	75	73	51	29	19	16	9
攻 32k: n_ctx15 N128 (final, seed42)	77	73	55	31	17	15	8
攻 32k: n_ctx15 N128 (final, seed1234)	79	71	59	31	18	15	10
攻 32k: N256 降压缩比 (final)	77	71	58	30	17	16	9

PG19 真长文蒸馏在 16k 稳定破天花板 (16 vs 13, +3)；攻 32k 的两条路线——加大训练窗口 (n_ctx15, 窗口 4096→8192) 与降低压缩比 (N256, 250:1→125:1) ——均为负结果 (32k 仍 ≈9)，跨 seed/ckpt 确认。

coef	8k	16k	32k
0.3	14	11	8
0.5	14	13	9
0.7	11	10	8
2.0	10	6	5

C LongBench W0 (F1) vs base 开卷基线

数据集	mem W0	base 开卷	保留率
narrativeqa	2.6	16.0	16%
qasper	5.2	13.9	38%
musique	3.5	7.0	50%
multifieldqa_en	12.3	24.9	49%
hotpotqa	6.5	9.8	66%
2wikimqa	9.2	12.2	75%
平均	6.56	14.0	47%

D 关键代码与脚本

- 架构: src/memory/mem_space/{config,layer,selector,memory_bank,l3_summary}.py

- token-mass: `--use_readout_mass_bias --readout_mass_coef` (commit 79ad265)
- self-study 蒸馏: `scripts/build_distill_cache.py + scripts/launch_distill_chunk512_AB.sh` (commit 19477fb / 8ea2969)
- 长度课程解耦 + 数据饥饿护栏: `--t2_curriculum` (commit dadc19f / 2c22d80)
- 缓存指纹 + 命中率 fail-fast: commit 0cf2c23 / 438b674
- 设计文档: `versions/v21_selfstudy_distillation.md`
- 阶段报告: `status/READOUT_ATTACK_STAGING_REPORT.md`
- 源码仓库: <https://github.com/liuhanzuo/Mixture-of-Memory>